Graphs in Motion

A Didactic Theory of Graph Representations, Small-Step GSLTs, and Compositional Cost

Oruži Zarathustra Goertzel

September 2026 — working draft

Abstract

A finite graph may be stored as an edge list, adjacency matrix, family of neighbour rows, finite neighbour sets, incidence matrix, or compressed sparse row structure. These carriers describe the same mathematical object but have different query, revision, conversion, and memory behaviour. We present a Lean 4 formalization that keeps those claims separate and then reconnects them compositionally. The declarative meaning of every simple-graph representation is a Mathlib $SG(\mathsf{Fin}n)$. A representation refinement is an actual construction together with a commuting theorem. We distinguish such extensional refinements from operational algorithms: wrapping an entire conversion function in one rewrite records an itinerary, but does not expose the conversion's internal computation. The first detailed conversion machine materializes an edge list as an adjacency matrix by consuming one stored edge occurrence per rewrite. Its executable step function is exactly equivalent to its relational semantics, every step preserves an exact completion invariant, and its terminal matrix is proved equal to the independent refinement. The same construction is authored as a nonempty sorted `LanguageDef`: its generic executor consumes one edge occurrence per `Write` rule and builds one persistent symmetric-update node, while an independent denotation commutes with the detailed matrix machine. The second detailed LanguageDef moves inside graph mathematics itself. It executes append, reverse, and homomorphic map on finite-graph walks, one edge at a time. Repeated endpoint indices and explicit adjacency queries make malformed joins and forged non-edges fail closed; proof-relevant executions terminate at exactly the corresponding Mathlib walk operations with closed linear step counts. From this same sorted constructor inventory and guarded rewrite relation we generate an endpoint-indexed native walk judgment. Every Mathlib walk inhabits that judgment, canonical encodings are injective, and identity, composition, reversal, and homomorphic map form a certified reachability proof theory whose operations retain exact GSLT executions. Semantic invariants are characterized as realizations into discrete semantic GSLTs. Resource use is additional proof-relevant structure, not something recoverable from a bare rewrite relation: costed GSLTs form a category, writer enrichment is functorial, and cost erasure is natural.

The theory includes positive and negative examples for six ordinary graph representations; exact refinement maps through a canonical matrix waist; the first occurrence-level conversion GSLT; semantic machines for revision and querying; a bridge to ordinary Mathlib matrices; break-even theorems for migration and maintained multi-view channels; worked query-centric, edge-centric, revision-heavy, and space-constrained migration decisions; semantic queries for hypergraphs, enriched metagraphs, and distinction graphs; and transport of connectedness, bipartiteness, acyclicity, degree, and Hamiltonicity. Ore's and Dirac's theorems are proved over Mathlib's graph semantics and then transported through every representation conversion. Raw native walk terms are characterized by unique dependent Mathlib walks, preserving exact length and repeated-edge evidence. A work-in-progress appendix marks the later arrows: detailed matrix-to-row and row-to-CSR machines, walk cancellation and incremental trail/path admission, an exact executable syntax and elaborator, certified SAT/DQBF refinement, and a CeTTa runtime realization. The result is both a graph library and a worked example of how a

mathematical library can expose operational meaning without confusing data layout, semantics, provenance, and cost.

Status legend. Solid blue arrows in the figures are implemented constructions; their labels distinguish detailed local rewrites from total refinement functions. No algorithmic granularity is inferred from colour alone. Solid green arrows are proved semantic interpretations or commuting laws; purple arrows carry selected resource grades. Orange dashed arrows are deliberately future work. Red dotted boxes are negative results: they delimit what cannot be reconstructed or preserved. This visual distinction is part of the mathematical claim, not decoration.

Contents

1 One graph, many clothes **3**

2 The declarative and operational layers **4**
2.1 The mathematical authority . . . 4
2.2 A graph presentation . . . 4
2.3 Refinement is a construction plus a theorem . . . 4
2.4 What a GSLT contributes . . . 5

3 The finite representation portfolio **5**

4 From refinement specifications to a detailed conversion GSLT **6**
4.1 The canonical matrix waist . . . 6
4.2 Occurrence-by-occurrence edge-list materialization . . . 7
4.3 The rewrite-bearing LanguageDef . . . 8
4.4 Soundness, exactness, and no invention . . . 9

5 Walk algebra as an operational LanguageDef **9**

6 Native types and reachability proofs **11**

7 What exactly is "the semantics"? **12**
7.1 Selected invariants . . . 12
7.2 Semantic squares compose . . . 13

8 Three motions that must not be conflated **13**

9 Semantic query and revision GSLTs **14**
9.1 A reusable semantic query machine . . . 14
9.2 Ordered mixed workloads . . . 14
9.3 Physical editing policy . . . 15

10 The ordinary matrix bridge **15**

11 Cost is selected structure, not hidden semantics **16**
11.1 A resource monoid . . . 16
11.2 Why a bare GSLT has no canonical cost . . . 16
11.3 When is migration worth paying for? . . . 17

11.4 Worked representation decisions 18

12 Higher graphs and controlled information loss **19**
12.1 Hypergraphs 19
12.2 Enriched metagraphs 19
12.3 Distinction graphs 20

13 Transporting ordinary graph theorems **20**
13.1 Hamiltonian cycles, Ore, and Dirac 20

14 When a LanguageDef has transformation semantics **22**

15 Why this organization scales beyond graphs **22**

16 Formal inventory and theorem map **23**

17 Conclusion **23**

A Work in progress: detailed transformation portfolio **25**

B Work in progress: from walks to path rewriting **25**

C Work in progress: general source language and compiler **26**

D Work in progress: cost, editing, and certified solving **26**

1 One graph, many clothes

Consider the path $0 - -1 - -2$. Figure 1 gives four of its concrete forms. The representations are not equal as data. They do, however, denote the same symmetric, irreflexive adjacency relation. This is the first distinction on which the development rests:

$$\text{concrete carrier} \quad \neq \quad \text{mathematical graph denoted by that carrier.}$$

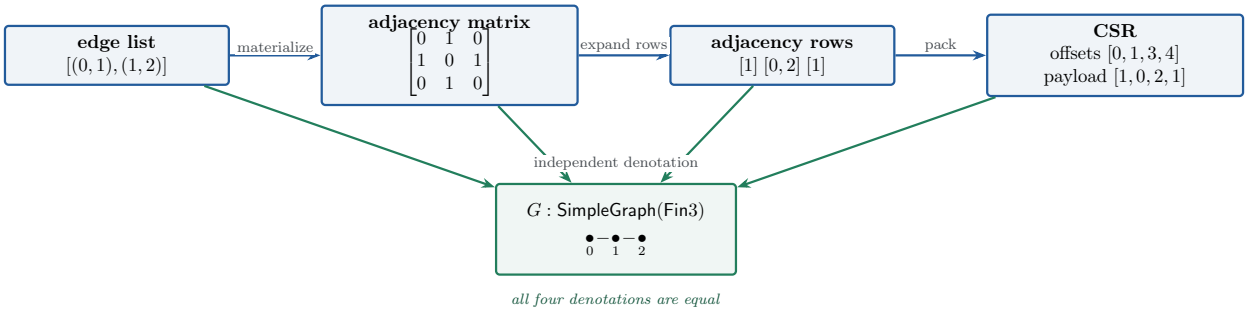


Figure 1: One graph in several operational forms. Blue arrows construct new data. Green arrows interpret data as a standard mathematical graph. A conversion is accepted only after the triangle with the independent meaning commutes.

The elementary slogan—"one graph, many clothes"—already predicts three different kinds of motion:

1. *conversion* changes the clothes while preserving the graph;
2. *revision* inserts or erases an edge and therefore changes the graph;
3. *query execution* changes a request into an answer while preserving the world being queried and the answer selected from it.

Conflating these motions leads to false invariants, circular semantics, and misleading cost claims. Sections 4–9 separate their mathematical specifications and give small-step GSLTs only where the local transition structure has actually been defined.

2 The declarative and operational layers

2.1 The mathematical authority

For a fixed finite vertex type $\mathsf{Fin}n$, the declarative authority is Mathlib's $\mathsf{SimpleGraph}(\mathsf{Fin}n)$: a symmetric, irreflexive relation [1, 2]. This choice matters. The operational development does not mint a private replacement for ordinary graph theory. Standard notions such as connectedness, bipartiteness, acyclicity, neighbourhoods, degree, walks, and Hamiltonian cycles remain Mathlib predicates and objects.

2.2 A graph presentation

A finite graph *presentation* packages just enough to compare concrete representations:

$$\begin{aligned}
P.\mathsf{Carrier} &: \mathsf{Type},\\
P.\mathsf{denote} &: P.\mathsf{Carrier} \to \mathsf{SimpleGraph}(\mathsf{Fin}n),\\
P.\mathsf{edge} &: P.\mathsf{Carrier} \to \mathsf{Fin}n \to \mathsf{Fin}n \to \mathsf{Measured}(\mathsf{Bool}),\\
P.\mathsf{edge_sound} &: P.\mathsf{edge}(x,u,v).\mathsf{value} = \mathsf{true} \iff P.\mathsf{denote}(x).\mathsf{Adj}(u,v),\\
P.\mathsf{storageCells} &: P.\mathsf{Carrier} \to \mathbb{N}.
\end{aligned}$$

Here $\mathsf{Measured}(A) = A \times \mathbb{N}$ records a value and an exact structural work count. The count means inspected abstract cells or entries; it is not asserted to be nanoseconds, cache misses, or machine instructions.

The denotation is chosen independently of the observer. Thus edge_sound is evidence rather than a definition by fiat. This small directionality choice prevents an implementation bug from becoming "true" merely because the implementation returned it.

2.3 Refinement is a construction plus a theorem

Given presentations P and Q, a refinement is

$$f : P.\mathsf{Carrier} \to Q.\mathsf{Carrier} \qquad \text{with} \qquad \forall x, \quad [\![f(x)]\!]_Q = [\![x]\!]_P.$$

The function is load-bearing. Two separately written implementations that happen to agree on test outputs are not yet a refinement. Refinements have identities and compose in data-flow order, so finite-graph presentations form a category. Every refinement also constructs a *channel* $(x, f(x))$ whose two live views are certified to denote the same graph.

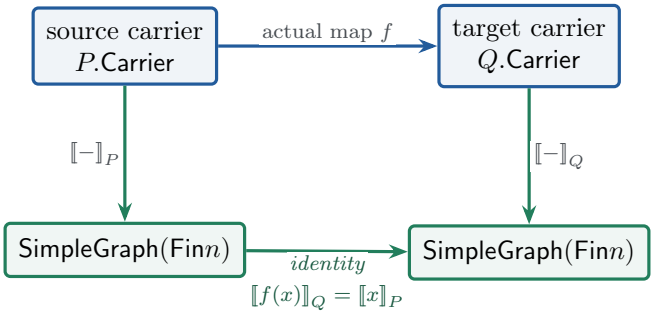


Figure 2: The refinement square. The upper arrow constructs target data; the lower comparison says that mathematical graph meaning is unchanged.

2.4 What a GSLT contributes

A graph-structured lambda theory is a triple $S = (T, E, R)$: terms, an equational setoid, and a rewrite relation respecting the equations [10, 5]. Its proof-relevant paths retain the ordered occurrences of primitive rewrites. In this development, a GSLT answers questions that a static commuting equation cannot:

- Which transformation was actually performed?
- In what order were edits and queries consumed?
- Which intermediate layouts existed?
- How many exposed local rewrite occurrences happened?
- What can be reflected from an arbitrary target step or terminal state?

The graph meaning may stay fixed even when these operational facts differ.

There is an important granularity condition. If a rewrite merely applies a total function that performs the whole conversion, the resulting GSLT records that the function was invoked and supports path composition, but it does not yet explain the algorithm inside that function. We call such an arrow an *abstract refinement leg*. A *detailed operational GSLT* instead exposes the evolving accumulator or worklist, gives local rewrites, relates an executable successor to those rewrites, and proves that the terminal state equals an independently specified result. This paper reserves algorithmic step and cost claims for the latter.

3 The finite representation portfolio

The current portfolio contains six ordinary representations and one enriched embedding representation. Table 1 gives one verified strength and one verified limitation for each. The point is not that one layout is globally best. The point is that every advantage is indexed by an observation, and every conversion has a price.

Several qualifications are important. A finite-set membership request is one operation in the chosen interface; this does not claim that every concrete finite-set implementation is constant-time. Likewise, CSR is cache-friendly in ordinary implementations, but the formal result here concerns exact slices, payload sizes, and scans. A later physical realization must connect those structural operations to bytes, caches, and wall-clock measurements.

Representation	Positive control	Negative control
Edge list	Exact authored edge-occurrence enumeration; adjacency is a left-to-right scan.	An absent edge forces inspection of the entire list.
Adjacency matrix	An edge request takes one abstract cell probe and one query-GSLT step.	Even the empty graph occupies n^2 abstract cells.
Adjacency rows	The neighbour list of one vertex is returned exactly.	An absent neighbour forces a full scan of that vertex's row.
Neighbour finsets	Membership and common-neighbour intersection are native set operations.	Enumeration order and duplicate occurrences are erased.
Incidence matrix	A named vertex–edge incidence request takes one abstract cell probe.	Testing whether two vertices are adjacent may inspect every edge column.
CSR	A vertex row is an exact contiguous slice; storage is linear in offsets plus payload.	A missing target still forces a full scan of the selected row.
Rotation system	The clockwise successor around a vertex is represented directly.	Forgetting cyclic order is not injective: clockwise and counterclockwise embeddings may have the same underlying graph.

Table 1: Every representation earns a positive and a negative theorem. Work counts are structural costs in the selected abstract model, not unqualified hardware-complexity claims.

4 From refinement specifications to a detailed conversion GSLT

4.1 The canonical matrix waist

The library has total refinement functions connecting edge lists, adjacency matrices, adjacency rows, neighbour finsets, incidence matrices, and CSR. A canonical adjacency matrix is their common materialization waist:

$$\mathsf{cell}_x(u, v) := P.\mathsf{edge}(x, u, v).\mathsf{value}.$$

Soundness of the source edge observer proves symmetry, irreflexivity, and commutation with $P.\mathsf{denote}$. From a matrix the system can build rows, neighbour finsets, a serialized edge list, an incidence matrix, or CSR (via rows).

These functions are mathematically substantive: each constructs its target carrier and has a commuting theorem. The tagged `RepresentationGSLT` records their composition as abstract refinement legs. Its one-leg rewrites do *not*, however, expose the loop or accumulator used inside a conversion. Consequently the number of those legs is an itinerary length, not an algorithmic work count. The distinction is visible in Figure 3: only the first materializer currently has a detailed small-step semantics.

At the abstract-refinement level, every state reaches its canonical matrix in at most one leg. Every target other than CSR is constructed from the matrix in at most one further leg; CSR uses the row intermediate. Hence every supported conversion has an abstract path of at most three legs. For the path graph of Figure 1, the itinerary

$$\mathsf{edgeList} \longrightarrow \mathsf{matrix} \longrightarrow \mathsf{rows} \longrightarrow \mathsf{CSR}$$

has three refinement legs. No claim about three machine operations follows.

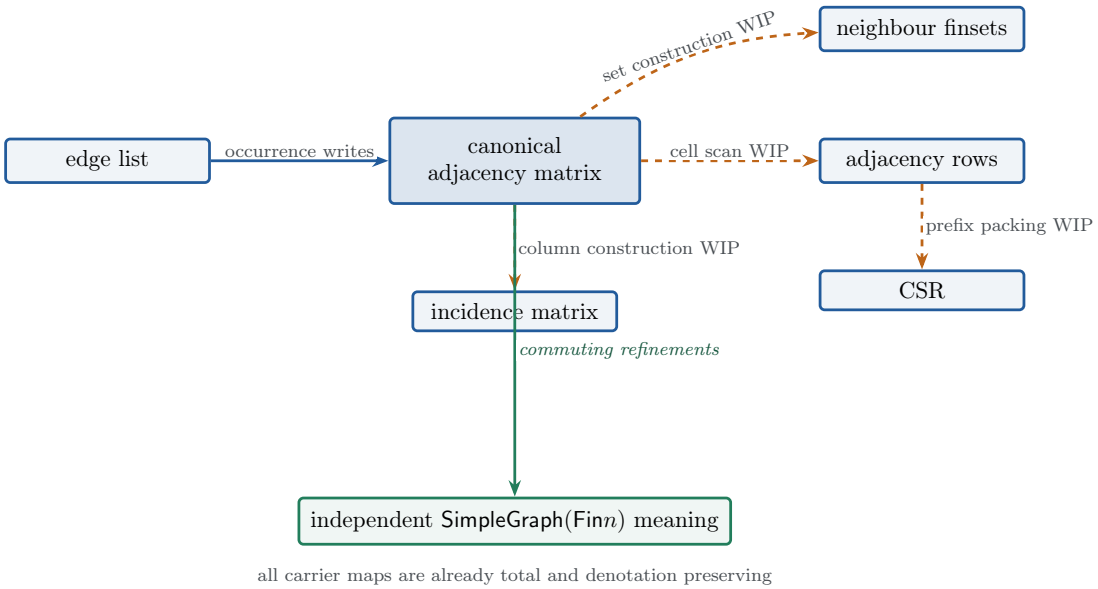


Figure 3: The matrix is a useful refinement waist, not the definition of graph meaning. All displayed conversions already have denotation-preserving total functions. The solid edge-list arrow additionally has a detailed small-step machine; dashed arrows mark the corresponding operational machines still to be constructed.

4.2 Occurrence-by-occurrence edge-list materialization

The detailed edge-list-to-matrix machine starts with an empty lawful Boolean matrix and retains the unconsumed authored edge occurrences:

$$\mathsf{active}(e_1 :: \cdots :: e_k, M) \qquad \text{or} \qquad \mathsf{done}(M).$$

One `write` rewrite consumes exactly the head occurrence and replaces M by a matrix whose addressed symmetric pair is set to true. A loop occurrence performs no update, matching simple-graph denotation. Duplicate occurrences remain visible as separate rewrites even though Boolean insertion is idempotent. Once the list is empty, one `finish` rewrite exposes the accumulator:

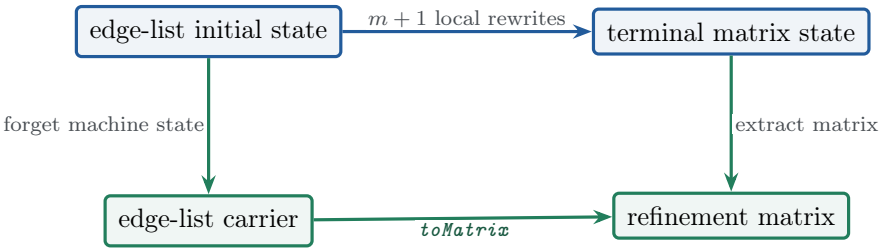
$$\begin{aligned} \mathsf{active}(e :: es, M) &\longrightarrow \mathsf{active}(es, \mathsf{insertOccurrence}(M, e)), \\ \mathsf{active}([], M) &\longrightarrow \mathsf{done}(M). \end{aligned}$$

This state transition is not only relational. An executable `step?` is proved to return `some target` exactly when the rewrite relation holds. The independently defined completion function folds all remaining occurrences into the current accumulator. Every local rewrite preserves that exact matrix:

$$s \longrightarrow t \quad \Longrightarrow \quad \mathsf{completion}(s) = \mathsf{completion}(t).$$

For a carrier with m stored occurrences, the constructed execution path has exactly $m + 1$ rewrites: one write per occurrence and one finish. Its terminal matrix is equal as a carrier, cell for cell, to the pre-existing extensional refinement `Transformations.toMatrix`. This is the commuting square

required of a genuine algorithmic realization:



Positive and negative controls make the rules load-bearing. The two-edge path takes three rewrites and contains the cells $0--1$ and $1--2$; the non-edge $0--2$ stays false. A one-vertex carrier containing only a loop materializes to the empty matrix. These examples distinguish occurrence consumption, no-invention, and loop erasure.

4.3 The rewrite-bearing LanguageDef

The detailed state machine also has an authored source in the shared `LanguageDef` format. It declares five sorts and nine sorted constructors:

$$\begin{aligned}
\mathsf{VertexZero} &: \mathsf{Vertex} & \mathsf{VertexSucc} &: \mathsf{Vertex} \to \mathsf{Vertex}\\
\mathsf{EdgeValue} &: \mathsf{Vertex}^2 \to \mathsf{Edge} & \mathsf{EdgesNil} &: \mathsf{Edges}\\
\mathsf{EdgesCons} &: \mathsf{Edge} \times \mathsf{Edges} \to \mathsf{Edges} & \mathsf{MatrixEmpty} &: \mathsf{Matrix}\\
\mathsf{MatrixSetSymmetric} &: \mathsf{Matrix} \times \mathsf{Vertex}^2 \to \mathsf{Matrix} & \mathsf{Active} &: \mathsf{Edges} \times \mathsf{Matrix} \to \mathsf{State}\\
\mathsf{Done} &: \mathsf{Matrix} \to \mathsf{State}.
\end{aligned}$$

The actual definition is assembled as a five-field `LanguageDef`; the only operational authority is its two explicit rewrite rows:

```
def language : LanguageDef :=
  LanguageDef.ofCore "EdgeListToMatrix" types terms []
    [writeRule, finishRule]

Write:
  Active(EdgesCons(EdgeValue(Source, Target), Rest), Matrix)
    ~> Active(Rest,
         MatrixSetSymmetric(Matrix, Source, Target))

Finish:
  Active(EdgesNil(), Matrix) ~> Done(Matrix)
```

There is no constructor named `convert`, `route`, or `via`, and neither rewrite invokes a host conversion function. The generic `LanguageDef` executor matches and instantiates these rows directly. For every canonical typed state (s), the formalization proves the exact list equality

$$\mathsf{rewriteOnce}(\mathsf{encode}(s)) = \mathsf{map}(\mathsf{encode}, \mathsf{toList}(\mathsf{step?}(s))).$$

Thus a successful typed action is an actual reduction generated from the authored definition, and a `Done` state has an empty successor list.

The update syntax has an independent interpretation:

$$\begin{aligned}
[\![\mathsf{MatrixEmpty}]\!] &= \mathsf{empty},\\
[\![\mathsf{MatrixSetSymmetric}(M,u,v)]\!] &= \mathsf{insertOccurrence}([\![M]\!],(u,v)).
\end{aligned}$$

Every source rewrite commutes with the independently defined detailed materializer, and folding the generated update expression denotes exactly `Transformations.toMatrix`. This gives the definition operational content at the persistent algebraic-update level. It does not yet claim that `MatrixSetSymmetric` is a mutable row-major implementation; decomposing that node into address calculation and concrete cell writes remains a lower physical refinement in Appendix C.

4.4 Soundness, exactness, and no invention

For every abstract refinement leg $s \longrightarrow t$, the theorem

$$[\![s]\!] = [\![t]\!]$$

is proved. It lifts to arbitrary execution paths and, in particular, to the total abstract conversion path. Pointwise adjacency therefore satisfies

$$[\![s]\!].\mathsf{Adj}(u, v) \quad \longleftrightarrow \quad [\![\mathsf{convert}(s, L)]\!].\mathsf{Adj}(u, v).$$

This is stronger than testing a few outputs: the target cannot invent or erase an edge. It is weaker than carrier invertibility, intentionally. Edge-list order, duplicate occurrences, matrix layout, and resource history may be lost. For the detailed materializer, exact endpoint equality strengthens this denotational theorem and shows that the small-step machine implements the same carrier function used by the refinement layer.

5 Walk algebra as an operational LanguageDef

Representation conversion is not the only place where graph mathematics has useful operational structure. A Mathlib walk is already an indexed object: $\mathsf{Walk}_G(u, v)$ carries its start and end vertices, and every constructor carries evidence that its next pair is adjacent. This makes walk append, reverse, and homomorphic map a particularly clean second LanguageDef vertical.

The authored language has five sorts, Vertex, WalkNode, Walk, Frames, and State, and thirteen rewrite rows. Its definition is not a wrapper around the three Mathlib functions:

```
def language : LanguageDef :=
  LanguageDef.ofCore "FiniteGraphWalkOperations"
    types terms []
    [ AppendStart, AppendScanCons, AppendScanNil,
      AppendRebuildCons, AppendRebuildNil,
      ReverseStart, ReverseScanCons, ReverseScanNil,
      MapStart, MapScanCons, MapScanNil,
      MapRebuildCons, MapRebuildNil ]

AppendStart:
  AppendRequest(WalkValue(S, M, Left), WalkValue(M, T, Right))
    ~> AppendScan(S, M, T, Left, WalkValue(M, T, Right),
         FramesNil(S))

AppendScanCons [SourceAdjacent(Current, Next)]:
  AppendScan(Current, M, T,
    WalkCons(Current, Next, M, WalkValue(Next, M, Tail)),
    Right, Frames)
      ~> AppendScan(Next, M, T, Tail, Right,
```

```
           FramesCons(Next, Current, Frames))

ReverseScanCons [SourceAdjacent(Next, Current)]:
  ReverseScan(Current, T, O,
    WalkCons(Current, Next, T, WalkValue(Next, T, Tail)),
    Accumulator)
      ~> ReverseScan(Next, T, O, Tail,
           prepend(Next, Current, O, Accumulator))

MapScanCons
  [ SourceAdjacent(SourceCurrent, SourceNext),
    MapVertex(SourceNext, MappedNext),
    TargetAdjacent(MappedCurrent, MappedNext) ]:
  MapScan(SourceCurrent, SourceTarget, MappedOrigin,
          MappedCurrent, SourceCons, Frames)
    ~> MapScan(SourceNext, SourceTarget, MappedOrigin,
         MappedNext, SourceTail,
         FramesCons(MappedNext, MappedCurrent, Frames))
```

Here `prepend` abbreviates the displayed nested `WalkValue`/`WalkCons` constructor expression in the source; it is not an external operation. The complete definition spells that expression out. The scan states accumulate reversed edge frames, and the rebuild rows pop one frame per step. Append and map therefore reveal both passes. Reverse needs only its scan accumulator.

The relation environment is deliberately small. It answers one source-edge query, one target-edge query, or one vertex image under the selected graph homomorphism. It never receives or returns a complete walk. For fully bound adjacency queries, the formalization proves exact preservation of the binding fibre. A vertex-image query introduces exactly its one output binding. The three-premise map row then checks the source edge, obtains the next image, and checks the resulting target edge in that order.

For endpoint-compatible walks $p : \mathsf{Walk}_G(a, b)$ and $q : \mathsf{Walk}_G(b, c)$, and a homomorphism $f : G \to H$, the generated proof-relevant paths have the exact endpoints

$$\begin{aligned} \mathsf{AppendRequest}(p, q) &\longrightarrow^* \mathsf{WalkDone}(p +\!\!+ q), \\ \mathsf{ReverseRequest}(p) &\longrightarrow^* \mathsf{WalkDone}(p^{\mathsf{rev}}), \\ \mathsf{MapRequest}(p) &\longrightarrow^* \mathsf{WalkDone}(\mathsf{map}(f, p)). \end{aligned}$$

These endpoints are the ordinary Mathlib operations, not a second private definition. If $\ell(p)$ is the edge length, the retained path lengths are exactly

$$2\ell(p) + 3, \qquad \ell(p) + 2, \qquad 2\ell(p) + 3$$

for append, reverse, and map respectively. Thus the cost statement counts actual authored rewrite occurrences. It is not a claim about elapsed time or memory traffic.

The three-vertex path $0 - -1 - -2$ supplies both polarities. Appending $0 - -1$ to $1 - -2$ reaches the two-edge Mathlib walk in five rewrites; reversing that walk takes four, and mapping it along the identity homomorphism takes seven. Conversely, an append request whose first endpoint is 1 and second start is 2 has no successor. A manually forged `WalkCons(0,2,...)` also has no successor because $0 - -2$ is not an edge. The endpoint repetition and adjacency premise are therefore load-bearing parts of the semantics.

6 Native types and reachability proofs

The operational walk language also generates a useful type theory. This is not a handwritten predicate renamed "native." Its static component is the generic executable sort checker instantiated with the complete constructor table of Section 5. Its dynamic component is the finite-path modality generated from the actual guarded rewrite relation.

For any GSLT S, the formalization constructs the proof-relevant span whose edges are values of $S.\mathsf{RewritePath}(s,t)$. Pullback and dependent sum along that span yield modalities

$$\Diamond_S^* P(s) \quad \Longleftrightarrow \quad \exists t,\ \mathsf{RewritePath}_S(s,t) \times P(t),$$

and, for every k, an exact-length modality $\Diamond_S^k$. Each diamond has its generated right adjoint; the corresponding Galois connections are proved once in the generic framework rather than separately for walks.

For a graph G and vertices u, v, the native walk predicate is the intersection of three source-derived conditions:

```
NativeWalk(G, u, v, term) :=
  grammarCheck(language, term, Walk)
  and term = WalkValue(u, v, node)
  and DiamondStar(theory(G),
        is WalkDone(WalkValue(v, u, _)),
        ReverseRequest(term))
```

The first line comes from the authored constructor signature. The second is the fibre of the authored `WalkValue` constructor. The third pulls a finite-path behavioral type back along the authored `ReverseRequest : Walk -> State` constructor. It therefore sees every `SourceAdjacent` premise used while scanning the candidate walk. Mathlib's `SimpleGraph.Walk` does not occur in this definition.

The executable checker is itself subject to inversion theorems. Successful argument checking consumes exactly one argument per authored parameter; a checked `WalkValue` exposes a checked `WalkNode`; and every checked walk node has outer form exactly `WalkNil` or `WalkCons`. These facts are derived from the constructor table rather than duplicated in a graph-specific parser. Small constructor-typing lemmas serve both canonical walks and deliberately forged examples.

The adequacy theorem now characterizes arbitrary raw terms:

$$\mathsf{NativeWalk}(G,u,v,t) \quad \Longleftrightarrow \quad \exists! p : \mathsf{Walk}_G(u,v), \quad t = \mathsf{encode}(p).$$

The forward reconstruction inverts the actual guarded reversal path. Matching exposes either the nil row, whose repeated endpoint must coincide, or the cons row, whose adjacency query decodes an in-range next vertex and certifies the edge. With an arbitrary checked accumulator a, induction on the remaining execution reconstructs a suffix p and identifies the terminal result as $\mathsf{WalkDone}(p^{\mathsf{rev}} +\!\!+ a)$. The converse combines constructor typing with the retained canonical reversal path; encoding injectivity supplies uniqueness. Neither the native predicate nor its rewrite rules contain an added host-language walk decoder.

The distinction between static and dynamic typing is load-bearing. On the three-vertex graph $0 - -1 - -2$, a manually formed `WalkCons(0,2,...)` passes the generated constructor-sort checker: its arguments really do have sorts `Vertex`, `Vertex`, `Vertex`, and `Walk`. It nevertheless fails the native type. Path decomposition shows that its reversal request reaches a scanner state, after which no successor is licensed because `SourceAdjacent(2,0)` is false. Conversely, an empty walk at

vertex 0 fails the $0 \to 1$ fibre before dynamics are considered. Thus constructor sorting, endpoint indexing, and operational adjacency are three distinct judgments with distinct negative controls. Further controls reject a sort-correct out-of-range interior vertex, a sort-correct tail with inconsistent endpoints, and a malformed nil arity.

For library use, a *native reachability proof* pairs a Mathlib walk with its generated native certificate. Exact reconstruction gives an equivalence between this existing proof carrier and the subtype of all accepted raw terms, with both round trips proved. These proof objects support

$$\begin{aligned} \mathsf{id}_u &: u \longrightarrow u, \\ q \circ p &: u \longrightarrow w, \\ p^{-1} &: v \longrightarrow u, \\ f_* p &: f(u) \longrightarrow f(v). \end{aligned}$$

Identity and composition satisfy the category laws; reversal is involutive and reverses composition order; homomorphic map preserves identity and composition and respects composition of graph homomorphisms. These are proved by transporting Mathlib's walk laws through the injective canonical encoding. They are also computational statements: composition, inversion, and map inhabit the exact-length path modalities with the work counts proved in Section 5. Closed walks are already the diagonal native fibres $\mathsf{NativeWalk}(G, u, u)$.

The equivalence transports every proof observer to accepted syntax. Applying the generic observer-factorization transport law retains the distinction between the two-edge direct walk and a four-edge detour at the same endpoint fibre: exact length is preserved by native encoding and lost by endpoint-only erasure. Reversal is involutive, but a walk followed by its reverse retains its edges; it is not identified with the empty walk. Trail and path refinements will be subtypes only after their own incremental admission machines exist.

7 What exactly is "the semantics"?

7.1 Selected invariants

For a GSLT S, a semantic invariant with values in M is an independently chosen map $d : S.T \to M$ preserving equations and rewrites. The library proves preservation along execution paths and reflexive–transitive execution, and restricts the operational system to any semantic fibre $d^{-1}(m)$.

Categorically, this is exact rather than metaphorical:

Theorem 1 (Invariants as discrete realizations)**.** *For every operational GSLT S and semantic carrier M, semantic invariants $S \to M$ are equivalent to operational realizations*

$$S \longrightarrow \mathsf{Discrete}(M),$$

where each source step is sent to the unique zero-length semantic path licensed by equality of its endpoint denotations.

Thus representation conversion is interpreted into a discrete graph GSLT. This does *not* say that all graph semantics are static. Revisions need a non-discrete semantic target, developed in Section 9.

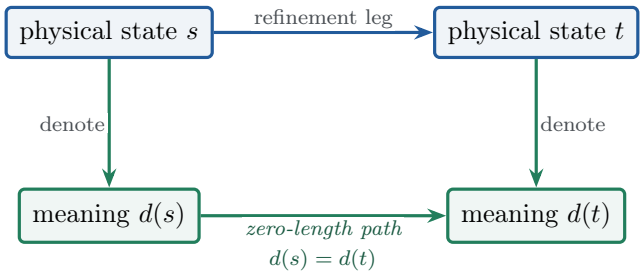


Figure 4: A semantically silent refinement leg. The source still records a real construction occurrence and may carry a selected grade; only its graph meaning is unchanged. Internal algorithm steps require a detailed machine.

7.2 Semantic squares compose

An operational realization may expand one source step into a finite target path. A *denotation square* adds independently selected source and target meanings and a map between them. Such squares compose in the same order as their operational realizations. This is the reusable answer to "does the whole pipeline commute?": each new arrow extends the end-to-end square rather than merely accumulating pairwise output agreements.

The formalization also proves two useful negative controls:

- stage or layout name is not invariant under representation conversion;
- request/answer phase is not invariant under query execution.

The surrounding mathematical theory must say which observation is semantic. A GSLT does not arbitrarily decree that every state field is meaning.

8 Three motions that must not be conflated

Figure 5 is the conceptual centre of the paper.

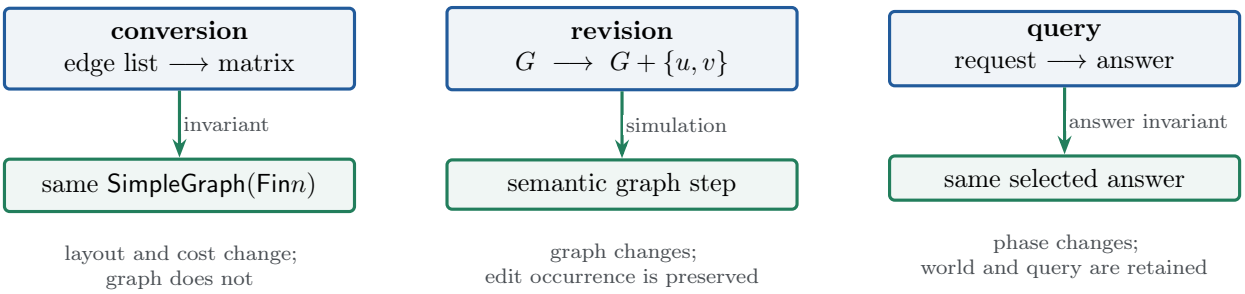


Figure 5: The three semantic shapes. Conversion is a realization into a discrete graph theory; revision maps to a genuine semantic graph step; query maps to a semantic request/answer step and then to a discrete answer.

9 Semantic query and revision GSLTs

9.1 A reusable semantic query machine

Given an independent observation function $o : W \to Q \to A$, the generic semantic query GSLT has states

$$\mathsf{request}(w, q) \qquad \text{and} \qquad \mathsf{answer}(a),$$

and exactly the step

$$\mathsf{request}(w, q) \longrightarrow \mathsf{answer}(o(w, q)).$$

Its terminal theorem states that every reachable normal answer is exactly $o(w, q)$; a separate theorem excludes any different answer. The selected answer is itself a semantic invariant, hence an operational realization into the discrete GSLT on A.

For adjacency matrices, hypergraphs, metagraphs, and sampled distinction graphs, the concrete query GSLT first translates to this semantic machine and then to a discrete Boolean answer. The factorization is visible:

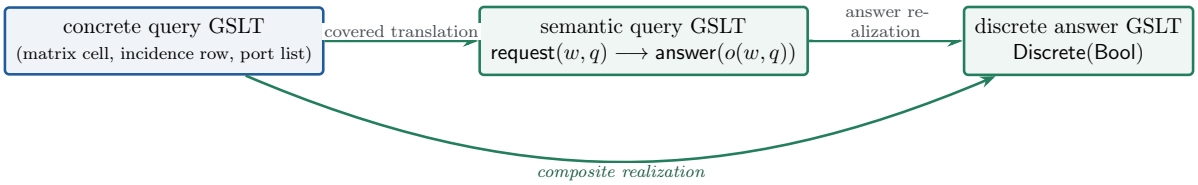


Figure 6: Concrete execution does not define truth. It is translated to an independent semantic request/answer machine, which in turn conserves the exact answer. Covered reverse theorems provide the no-invention boundary.

9.2 Ordered mixed workloads

An editable presentation adds a physical edit operation and proves that it implements an independently defined semantic edit $\mathsf{applyMeaning} : \mathsf{SimpleGraph}(\mathsf{Fin}n) \to \mathsf{EdgeEdit} \to \mathsf{SimpleGraph}(\mathsf{Fin}n)$. The mixed workload machine consumes authored edits and queries in order. Its state retains the current concrete graph, remaining workload, reverse answer accumulator, and resource account. Duplicated queries remain duplicated answer occurrences.

The independent semantic revision GSLT has no layout or cost field:

$$\begin{aligned}
&\mathsf{active}(G,\ \mathit{work},\ \mathit{answers}) && \longrightarrow\ \mathsf{active}(\mathsf{applyMeaning}(G, e),\ \mathit{rest},\ \mathit{answers}),\\
&\mathsf{active}(G,\ \mathsf{query}(u, v) :: \mathit{rest},\ A) && \longrightarrow\ \mathsf{active}(G,\ \mathit{rest},\ \mathsf{decide}(G.\mathsf{Adj}(u, v)) :: A),\\
&\mathsf{active}(G, [], A) && \longrightarrow\ \mathsf{finished}(G, \mathsf{reverse}(A)).
\end{aligned}$$

The concrete-to-semantic state map applies the presentation denotation and drops physical resource/layout fields, but preserves workload order and answer occurrences.

One-step commutation is strengthened to a covered equivalence: from an interpreted concrete source, every semantic step is exactly the interpretation of a concrete successor. Full execution then yields the same final graph and ordered answers as semantic execution. Positive and negative controls show that insertion really changes adjacency and therefore no graph-denotation invariant can exist for that step.

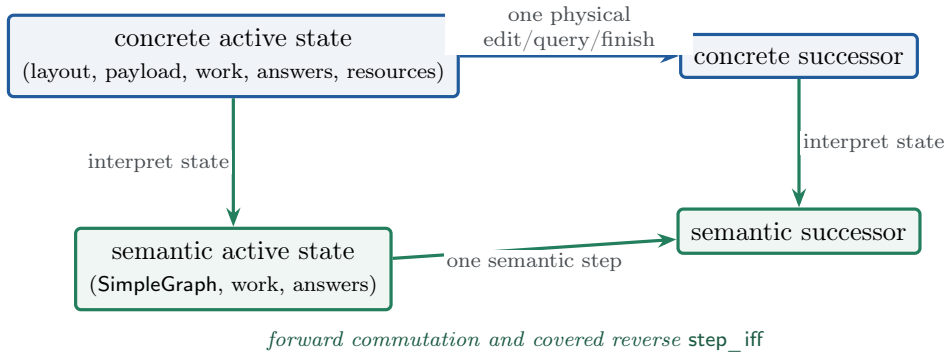


Figure 7: Revision is not forced into a false invariant. Concrete insertion maps to semantic insertion, and covered semantic successors reconstruct a concrete successor. Full execution agrees with the semantic ordered fold.

9.3 Physical editing policy

The adjacency matrix edit changes two symmetric cells and is structurally in-place in the chosen model. Other layouts currently use a correct baseline: canonicalize to a matrix, perform the edit, and materialize the original layout. Their positive theorem is semantic correctness; their negative theorem says that a rebuild with positive target storage is not in-place. These are intentionally not advertised as optimized update algorithms. The interface permits future persistent, differential, and local-update implementations to compete while preserving the same semantic square.

10 The ordinary matrix bridge

The graph adjacency-matrix carrier is connected to Mathlib's matrix type $\mathsf{Matrix}(\mathsf{Fin}n)(\mathsf{Fin}n)\,\mathsf{Bool}$. Its image is characterized by

$$\mathsf{IsSimple}(M) := \big(\forall u\, v,\ M_{uv} = M_{vu}\big)\ \wedge\ \big(\forall u,\ M_{uu} = \mathsf{false}\big).$$

Conversion to the ordinary matrix is injective; converting back from a matrix equipped with IsSimple is an exact round trip. Most importantly,

$$[\![A]\!].\mathsf{Adj}(u, v) \quad \longleftrightarrow \quad A_{uv} = \mathsf{true}.$$

The raw matrix query GSLT has its own request and answer states. Encoding a graph-matrix query is answer-blind, and arbitrary-target reflection proves that a raw matrix step from an encoded source has exactly the encoded graph successor. Positive and negative canaries check a genuine path edge and a false diagonal cell.

This bridge is the natural entry point for future matrix-GSLT algorithms: Boolean matrix multiplication, transitive closure, semiring path problems, or linear-algebraic graph kernels can state their mathematical correctness over standard matrices and their operational behavior as GSLT realizations.

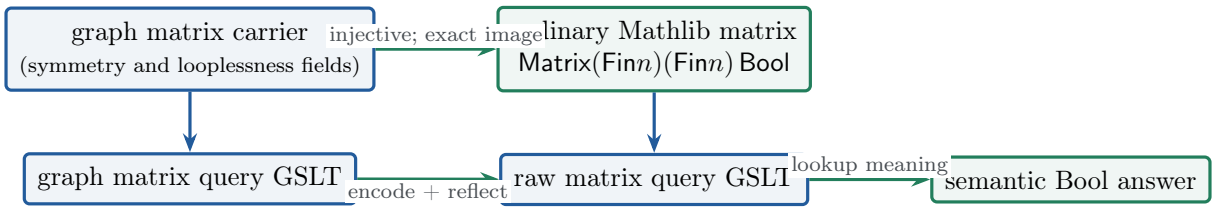


Figure 8: The matrix representation participates in ordinary matrix mathematics and in matrix GSLTs. The graph semantics, matrix semantics, and query execution agree without identifying their carriers.

11 Cost is selected structure, not hidden semantics

11.1 A resource monoid

The graph resource account has five coordinates:

$$r = (\mathit{time}, \mathit{reads}, \mathit{writes}, \mathit{allocated}, \mathit{peakTemporary}).$$

Sequential composition adds the first four coordinates and takes the maximum of peak temporary space, reflecting non-overlapping temporary lifetimes:

$$(r \star s).\mathit{peakTemporary} = \max(r.\mathit{peakTemporary}, s.\mathit{peakTemporary}).$$

The zero account is a unit and $\star$ is associative. Persistent construction has the abstract peak envelope

$$S_{\text{source}} + S_{\text{target}} + S_{\text{temp}},$$

whereas a destructive lower envelope is

$$\max(S_{\text{source}}, S_{\text{target}}) + S_{\text{temp}}.$$

The latter is only a lower-envelope model: a concrete algorithm must prove that it attains it. Structural in-place execution means zero allocation and zero temporary cells, not merely low time.

11.2 Why a bare GSLT has no canonical cost

A rewrite relation is proposition-valued, and the same local step can lawfully receive different grades under different physical models. The formal negative canary grades one step by one unit in one model and two units in another. Therefore there is no canonical cost endofunctor on bare GSLTs.

The correct domain is the category whose objects are pairs (S, γ), where S is a GSLT and γ is a selected monoidal grading of its steps. Morphisms preserve both operational steps and grades. Writer enrichment is functorial on this category, and erasing the accumulated writer account is a natural transformation to the underlying operational theory [6].

For the abstract representation-path GSLT, proof-relevant grades cover all ten refinement legs. Route accounts compose exactly; recursive execution agrees with the closed monoidal account; and erasing a costed route returns the original abstract path, not merely a path with the same endpoint. These facts are categorical properties of a *selected* grading. They do not derive an algorithmic cost for a conversion whose internal machine has not yet been exposed. Composing cost erasure with graph denotation yields the chain

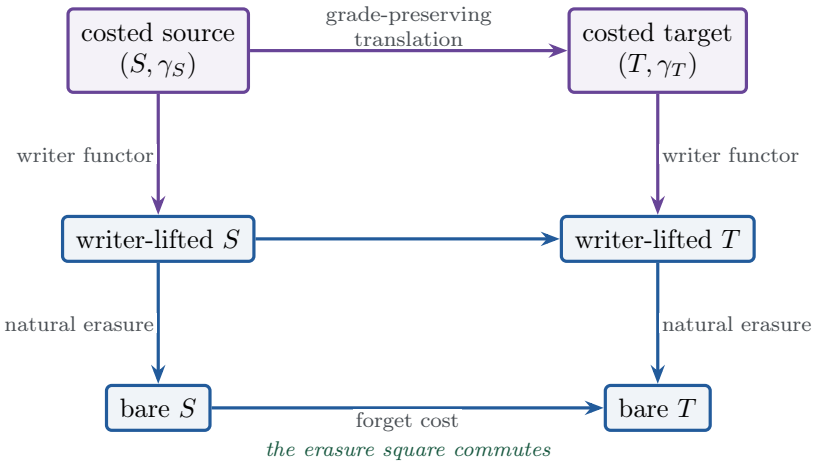
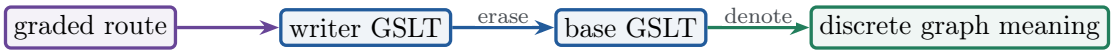


Figure 9: Costed operational theories. Writer enrichment is functorial only after a grading has been selected; natural erasure recovers the exact ordered base execution.



The same graph can carry different resource accounts, so resources cannot be reconstructed from graph meaning.

The detailed edge-list materializer supplies the first conversion grade with a direct operational basis: a path contains one local write rewrite per stored occurrence plus one finish rewrite. Translating these structural events into memory traffic or elapsed time still requires a separately justified physical model. For the remaining refinement legs, the present grades should be read as explicit abstract assumptions used in break-even algebra, not as costs extracted from hidden implementations.

11.3 When is migration worth paying for?

Suppose a source query costs $t+s$, a target query costs t, conversion costs c, and the future workload has q queries. Exact arithmetic gives

Theorem 2 (Static break-even)**.**

$$c + qt < q(t+s) \quad \longleftrightarrow \quad c < qs.$$

No asymptotic hand-waving is required. Weighted finite workloads similarly support exact average-case numerators, and pointwise dominance implies dominance for every such weighted workload.

If both representations are retained as a coherent channel and each of e edits costs an extra target synchronization u, then:

Theorem 3 (Dynamic multi-view break-even)**.**

$$\textit{maintain the second view faster} \quad \longleftrightarrow \quad c + eu < qs.$$

Without query savings, every genuinely costly channel is slower. A time-winning channel may still be inadmissible because its two retained carriers exceed the space budget.

This is the formal analogue of the representation portfolios and channelling constraints used by Essence and Conjure [3, 4]. The difference is that the representation maps and semantic coherence laws are checked, while the cost policy remains explicitly separate from truth.

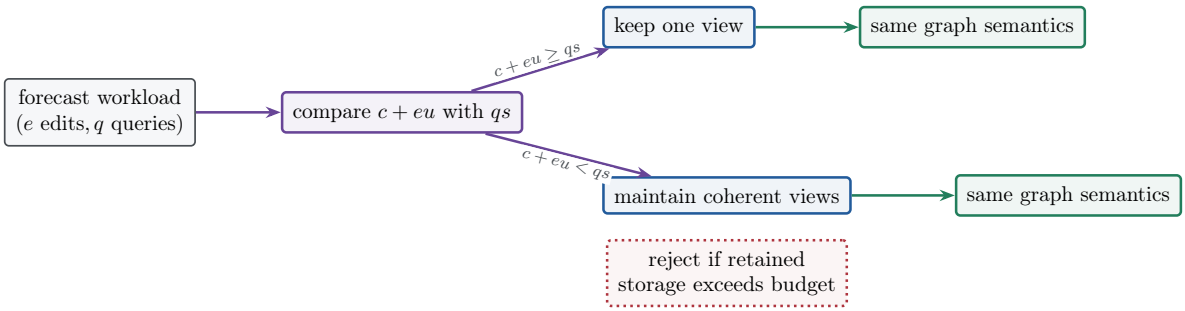


Figure 10: Representation choice is a resource policy over already proved semantic alternatives. The policy may select a faster view; it cannot decide whether an edge exists.

11.4 Worked representation decisions

Constraint modelling makes the workload dependence concrete. Essence lets a problem be stated over an abstract combinatorial object; Conjure refines such objects and expressions through representation choices, and can retain multiple channelled representations [3, 4]. Our graph examples isolate the corresponding decision after semantic correctness has already been proved. The unit below is a *structural event*, not a timing measurement: one edge occurrence consumed, one edge-list occurrence probed, one matrix cell read, or one synchronized occurrence update.

Pairwise graph-isomorphism constraints: migrate. Consider one side of a graph-isomorphism model that ranges over every unordered vertex pair and constrains whether adjacency is preserved by the candidate bijection. A 200-vertex simple graph with 800 distinct edges has $\binom{200}{2} - 800 = 19{,}100$ absent pairs. Those absent probes alone force 19,100 complete scans of an 800-occurrence edge list. The detailed materializer costs $800 + 1 = 801$ rewrites and the matrix then costs one read per probe:

$$\begin{aligned} \mathsf{stay} &= 19{,}100 \cdot 800 = 15{,}280{,}000, \\ \mathsf{migrate} &= 801 + 19{,}100 \cdot 1 = 19{,}901. \end{aligned}$$

The Lean theorem proves the strict inequality. Present-edge probes can only add further nonnegative savings relative to this absent-pair subworkload, so the decision does not depend on optimistic early matches.

Edge-driven graph colouring: do not migrate. In the standard colouring model, one inequality is emitted per authored edge:

```
forAll (u,v) in edges(G) . colour[u] != colour[v]
```

Streaming the 800 edge occurrences costs 800 events. Materializing the matrix and using the selected full scan schedule over its 200^2 ordered cells costs $801 + 40{,}000 = 40{,}801$. Here the edge list already presents exactly the iteration domain required by the constraints. The negative theorem is as important as the positive one: a matrix is not intrinsically the "better" representation.

A revised sparse graph: forecast both edits and reads. For a 1,000-vertex, 3,000-occurrence graph, initial materialization costs 3,001 events, the edge list occupies 6,000 endpoint cells, and the matrix occupies 1,000,000 Boolean cells. With 25 edits and ten absent-edge queries, selecting one event for both the source update and target synchronization, the exact accounts are

$$\mathsf{channelled} = 3{,}001 + 25(1 + 1) + 10 = 3{,}061, \qquad \mathsf{single} = 25 + 10 \cdot 3{,}000 = 30{,}025.$$

The maintained matrix wins on events. With 4,000 edits and one query, the ruling reverses:

$$\mathsf{single} = 7{,}000 < 11{,}002 = \mathsf{channelled}.$$

Finally, even the read-heavy winner is inadmissible under a 750,000-cell budget because retaining both views requires 1,006,000 cells. All three claims are checked concrete instances of the general break-even and space laws, not prose estimates. The unit edit and synchronization grades are explicit model choices; unlike construction and query counts, they are not yet extracted from a detailed edit implementation.

Workload	Source-only events	Alternative events	Ruling
19,100 absent-pair probes	15,280,000	19,901	materialize matrix
edge-driven colouring	800	40,801	stream edge list
25 edits, 10 absent probes	30,025	3,061	maintain both views
4,000 edits, 1 absent probe	7,000	11,002	keep edge list
read-heavy, 750,000-cell budget	6,000 retained	1,006,000 retained	reject channel

Table 2: Worked decisions in the selected structural-event model. The same semantically correct representations produce different optima because the constraint iteration domain, revision rate, query rate, and storage budget differ.

12 Higher graphs and controlled information loss

Ordinary simple graphs are only one semantic level. The same method is applied to three richer structures.

12.1 Hypergraphs

A finite hypergraph stores named edge occurrences with finite vertex supports [7]. The incidence query has a detailed linear-scan GSLT and an exact semantic query translation. Two standard projections are formalized as total maps:

- the *two-section*, connecting two vertices when some hyperedge contains both;
- the bipartite *Levi graph*, connecting vertex objects to edge objects by incidence.

The existing one-step Levi transition records application of the projection, but does not yet expose an occurrence-by-occurrence Levi construction. It is therefore classified here as an abstract refinement leg. A sharp negative example shows that one ternary hyperedge and three binary hyperedges can have the same two-section while retaining different hyperedge-occurrence counts. Hence the two-section preserves pairwise adjacency but cannot reconstruct grouping or occurrence identity.

12.2 Enriched metagraphs

The metagraph carrier gives each object an authored list of ports. A port has a target, label, and enrichment payload. Link querying is executable and proved against the corresponding relational meaning. Forgetting to an underlying hypergraph preserves selected incidence but loses port order, labels, and enrichment. Reordered authored ports therefore provide a negative witness: the underlying relational worlds agree while the metagraph worlds remain distinct.

12.3 Distinction graphs

A finite sampled distinction structure is mapped to an adjacency matrix. The refinement theorem characterizes every cell, and its abstract transition composes with matrix querying and semantic Boolean answering. A detailed row/cell construction machine remains work in progress. The source sample is retained in the state because a matrix records that a separator exists, not which semantic witness established it. Two witnessed tables can erase to the same matrix with different witnesses; no global inverse is claimed.

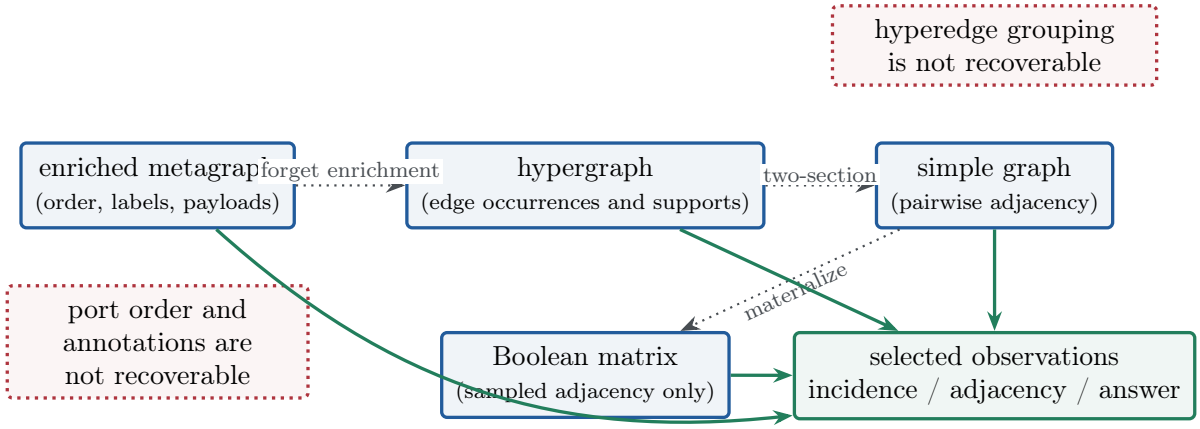


Figure 11: A hierarchy of useful forgetful maps. Each arrow preserves a named observation and has a negative theorem showing which richer structure is lost. Semantic adequacy is not misreported as representational invertibility.

Rotation systems fit the same picture: a cyclic order around each vertex is an enrichment of adjacency, not another lossless encoding of a plain graph.

13 Transporting ordinary graph theorems

Since every abstract refinement path preserves the exact Mathlib graph, *every* predicate on finite simple graphs transports. For

$$P : \mathsf{SimpleGraph}(\mathsf{Fin}n) \longrightarrow \mathsf{Prop},$$

the generic result is

$$P([\![s]\!]) \quad \longleftrightarrow \quad P([\![\mathsf{convert}(s, L)]\!]).$$

Specialized corollaries cover connectedness, bipartiteness, acyclicity, Hamiltonicity, and neighbour-set cardinality. Cost is deliberately absent: an absent edge in the path edge list takes two probes but one after matrix materialization, even though every extensional graph theorem is preserved.

13.1 Hamiltonian cycles, Ore, and Dirac

The declarative graph library contains a custom finite Hamilton-cycle certificate with an exact bridge to Mathlib's walk-based IsHamiltonianCycle: the constructed walk has the expected length, the expected vertex at each index, and visits every vertex in the required cyclic order.

Ore's theorem is proved by the standard longest-path rotation argument [9, 2]. For $|V| \geq 3$, if every distinct nonadjacent pair satisfies

$$|V| \leq \deg(u) + \deg(v),$$

then the graph is Hamiltonian. Dirac's theorem is derived as the specialization $|V| \leq 2\delta(G)$ [8]. The two operational transport theorems say: if the degree premise holds for the graph denoted by *any* source layout, then every materialized target layout denotes a Hamiltonian graph.

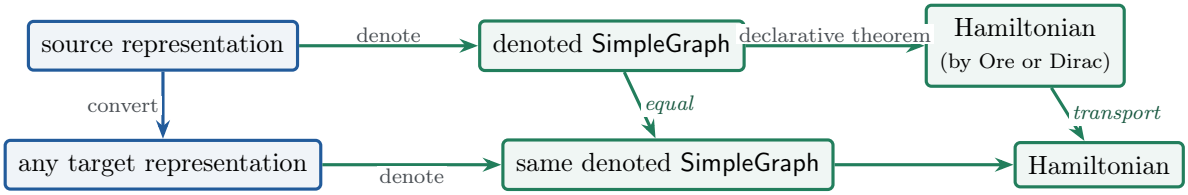


Figure 12: A declarative theorem is proved once over the standard graph and then transported through every representation refinement. This is property transport, not a claim that the conversion finds a Hamiltonian cycle efficiently.

The Chvátal–Erdős theorem is not part of the current formalization. An earlier unproved declaration was removed. A real proof requires a canonical separator/connectivity interface, a Menger-style bridge, and a longest-cycle argument; retaining the gap explicitly is more informative than preserving a name without a theorem.

14 When a LanguageDef has transformation semantics

An earlier draft included a validated `LanguageDef` that declared names for graphs, layouts, routes, plans, and commands while providing no equations or rewrites. That declaration has been removed from the active graph umbrella and from the claimed formal stack. Validation of a constructor vocabulary establishes only that the declaration is internally well formed; it does not determine graph payloads, route endpoints, conversion behavior, or execution.

The negative criterion is simple. If two incompatible conversion functions can be placed behind the same declared route name without changing the `LanguageDef`, then the declaration is not an authority for that conversion. Similarly, a term such as `via(plan, graph)` has no operational content until its graph encoding, typed plan elaboration, and rewrite behavior are derived from inspectable rules.

Sections 4.3 and 5 supply two positive replacements. The first encodes a complete conversion state, lets two rewrites determine every successor, and commutes with an independent matrix interpretation. The second gives thirteen local rules for three ordinary walk operations and proves exact correspondence with Mathlib. Both are genuine operational language definitions; neither delegates its complete operation to an external relation.

Section 6 adds a second test: a useful LanguageDef should support derived judgments that actually depend on its syntax and dynamics. The walk native type depends simultaneously on generated sort checking, constructor fibres, and successful guarded execution. Removing the adjacency premise changes the negative theorem, and changing the rewrite table changes the behavioral modality. The type theory is therefore downstream of the LanguageDef rather than administrative metadata placed beside it.

It is not yet a general graph-plan language. Such a language still needs a canonical graph payload codec, endpoint-safe typed plan elaboration, local rules for every supported algorithm, and a generated runtime realization. Plan composition may then elaborate into the existing indexed path category. That broader contract is stated in Appendix C; the selected materializer is its first proved vertical, not evidence that every abstract route already has detailed syntax and execution.

15 Why this organization scales beyond graphs

The same distinction applies to terms, formulas, proof states, and cognitive world models. A term may be represented as a tree, a DAG with shared subterms, an e-graph, a hash-consed heap, or a serialized array. These can denote the same syntax while having very different substitution, revision, sharing, and query costs. Rewriting the representation is semantically silent only relative to a selected meaning; beta reduction or belief revision changes that meaning and needs a semantic step.

The graph development therefore suggests a reusable recipe:

1. choose an established declarative object;
2. define concrete presentations and independent denotations;
3. make each conversion an actual refinement;
4. expose execution as a proof-relevant GSLT;
5. map conversion to discrete semantics, revision to semantic dynamics, and queries to semantic request/answer machines;

6. add cost only as selected grading over exposed local steps, or label it explicitly as an abstract model over refinement legs;

7. prove both a useful positive result and a discriminating negative result for every abstraction boundary.

For world-model calculi, this says that one world may have many physical representations, each with distinct revision and extraction behavior. A planner over Bayesian networks or a probabilistic logic engine over decision diagrams can then be studied as a representation-sensitive coGSLT-like interface without turning scheduling, probability tables, or cache layout into semantic truth.

16 Formal inventory and theorem map

Table 3 lists the principal formal components. Module names are given rather than unstable line numbers.

At the formalization snapshot described here, the graph-representation umbrella builds successfully with the operational matrix-construction and walk `LanguageDef`s included and the earlier empty graph vocabulary excluded. Focused audits of this tranche find no proof placeholders, theorem wishlists, native-decision shortcuts, or new project axioms. Axiom reports for the selected theorems remain within ordinary Lean foundations where required (propositional extensionality, classical choice, and quotient soundness). These facts concern the Lean objects above; they do not certify an as-yet-unwritten CeTTa runtime or solver connection.

17 Conclusion

The useful abstraction is not "the adjacency matrix is the graph" and not "all representations are interchangeable." It is a network of concrete presentations over a shared mathematical object, with proof-bearing constructions between them and explicit operational theories for the motions that matter.

In the resulting picture:

- Mathlib graph theory says what a graph and its properties mean;
- refinement functions say which target carriers are constructed and prove what those carriers denote;
- detailed GSLTs say how exposed local states transform;
- native types organize constructor shape, endpoint fibres, and behavioral reachability into derived mathematical judgments;
- semantic revision and query GSLTs say how graph meaning and observations evolve;
- operational realizations and denotation squares make the entire chain compose;
- costed theories say when a correct representation change is worth its time and space;
- negative theorems say exactly what every forgetful map loses.

That organization is useful for ordinary graph algorithms, but its larger value is methodological. It offers a prototype mathematical library in which declarative truth and operational behavior are both first-class, cleanly separated, and then reconnected by proofs.

Module family	Principal checked content
`Representation.Basic`	Presentation, refinement, category, channel, measured linear-probe GSLT.
`EdgeList`, `AdjacencyMatrix`, `AdjacencyRows`, `NeighborFinsets`, `IncidenceMatrix`, `CSR`	Six concrete carriers, independent denotations, sound edge observers, storage/work facts, positive and negative controls.
`Transformations`, `RepresentationGSLT`	Actual conversion maps, ten abstract refinement legs, path composition, total routes through the matrix waist, and adjacency no-invention.
`EdgeListToMatrixGSLT`	Occurrence-by-occurrence symmetric cell updates, executable/relational step equivalence, exact completion invariant, $m+1$-step path, exact equality with the matrix refinement, and positive and negative controls.
`EdgeListToMatrixLanguageDef`	Five sorts, nine sorted constructors, two operational rewrite rows, exact generic-executor/typed-step agreement, terminal rejection, a commuting interpretation into the detailed matrix machine, and exact endpoint denotation.
`GraphTheory.Walk.LanguageDef`, `GraphTheory.Walk.Operational`, `GraphTheory.Walk.Examples`	Five sorts, sixteen sorted constructors, thirteen recursive rewrite rows, exact append/reverse/map executions to Mathlib walks, closed path-length formulas, and endpoint-mismatch/non-edge rejection controls.
`OSLF.Framework.PathTypeSynthesis`, `OSLF.MeTTaIL.NonrecursiveReduction`	Generic proof-relevant finite-path and exact-length modalities, their Galois connections, path decomposition, and exact one-layer execution for congruence-free LanguageDefs.
`GraphTheory.Walk.NativeTypeTheory`, `GraphTheory.Walk.ProofTheory`, `GraphTheory.Walk.NativeTypeExamples`	Generated static/dynamic native walk judgment, unique reconstruction of arbitrary accepted terms, equivalence with dependent walk proofs, exact-work behavioral types, walk-category operations, involutive reversal, functorial map, and endpoint, non-edge, out-of-range and malformed-arity controls.
`GraphTheory.Walk.ObserverLicences`	Transport of every proof observer to accepted syntax; endpoint erasure retains endpoint observations and loses exact length, witnessed by the direct walk and a repeated-edge detour.
`GSLT.Core.SemanticInvariant`, `SemanticTransport`	Invariants, semantic fibres, equivalence with discrete realizations, denotation squares, composition and path commutation.
`SemanticRevisionGSLT`, `RevisionQuery`, `RevisionPortfolio`, `WorldModelGSLT`	Independent semantic query and revision dynamics, concrete translation, covered reflection, full execution agreement, editable portfolio and world-model extraction.
`MatrixBridge`	Exact image in ordinary Boolean matrices, round trip, injectivity, adjacency/cell theorem, answer-blind matrix query simulation and reflection.
`CostModel`, `GSLT.Core.CostedOperational`, `CostedRepresentationGSLT`, `MultiView`, `MigrationExamples`	Resource monoid, in-place/persistent envelopes, exact break-even laws, costed-theory category, writer functor, natural erasure, proof-relevant graded routes, channel costs, and worked positive/negative workload decisions.
`Hypergraph`, `Metagraph`, `DistinctionBridge`, `HigherGraphSemanticGSLT`, `RotationSystem`	Higher-graph transformations and semantic queries, plus explicit information-loss counterexamples.
`Hamiltonicity`, `HamiltonianDegree`, `Representation.SemanticTransport`	Hamilton-cycle/Mathlib bridge, longest-path proof of Ore and Dirac, and property transport through every layout.

Table 3: The formal stack. Declarative mathematics, operational rules, semantic interpretation, and resource grading occupy different modules and are connected by named theorems.

A Work in progress: detailed transformation portfolio

The edge-list materializer is the first detailed conversion, not evidence that the remaining refinement legs are already small-step algorithms. The next three machines form the most informative end-to-end chain:

1. **Matrix to adjacency rows.** Scan coordinates in canonical row-major order, append a target exactly when the matrix cell is true, retain the current row and completed rows, and prove exact terminal equality with `matrixToRows`. The loop invariant must identify the accumulator with the processed prefix of coordinates.

2. **Adjacency rows to CSR.** Consume one row item at a time, maintain offsets and payload, prove monotone and bounded offsets, and prove exact equality with `rowsToCSR`. Prefix-sum and payload actions, rather than a hidden packing function, provide the cost basis.

3. **Composed realization.** Compose edge-list materialization, row expansion, and CSR packing while retaining every local event. Prove that erasure yields the existing three-leg refinement itinerary and that the final CSR carrier is exactly the existing total conversion result.

Further detailed candidates include matrix-to-edge-list cell scanning, matrix-to-incidence column construction, occurrence-preserving Levi graph materialization, and row/cell construction for distinction matrices. Until their states, local rewrites, invariants, and endpoint theorems exist, their current one-step transitions remain classified as abstract refinement legs.

B Work in progress: from walks to path rewriting

The implemented walk machine deliberately stops at total append, reverse, and homomorphic map. These operations already expose recursion and relation queries, and their generated native types already support the category laws, involutive reversal, and functorial homomorphic map on canonical reachability proofs. They do not yet form a normalization theory of paths. The next mathematically substantive extension is the free-groupoid view of an undirected graph: vertices are objects, walks are composable arrows, reversal is inversion, and an immediately retraced edge may cancel.

A candidate local rule has the genuinely inspectable form

$$(u \to v); (v \to u); p \;\longrightarrow\; p,$$

with both adjacencies and all endpoints explicit. Before admitting that rule to the solid language, the formalization must prove termination of the chosen orientation, confluence or a characterized normal-form relation, compatibility with append, and correspondence with the intended quotient of Mathlib walks. It must also distinguish cancellation of immediate backtracking from the stronger and generally false claim that arbitrary cycles disappear.

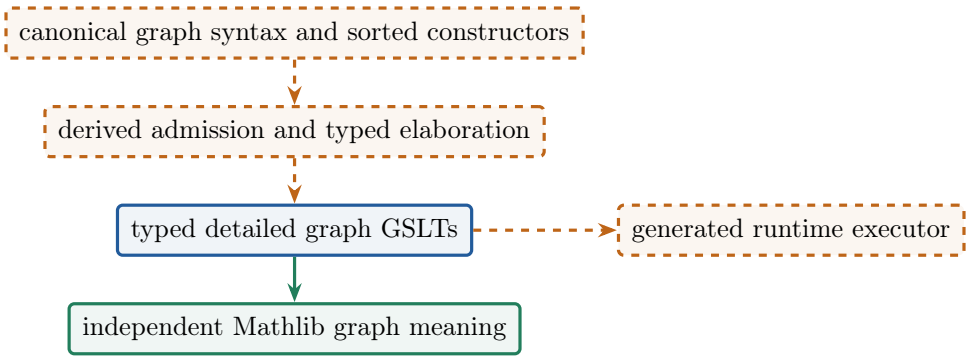
The present native type's raw-pattern exactness obligation is discharged in Section 6: every accepted term encodes one unique dependent Mathlib walk. Its reconstruction theorem uses the existing guarded scan rows and relation-query exactness. It does not introduce cancellation or identify distinct walks with the same endpoints.

Two further extensions should remain separate. An incremental trail/path admission machine can carry visited edge or vertex sets and reject the first duplicate occurrence; this operationalizes Mathlib's `IsTrail` and `IsPath` predicates without changing walk equality. A reachability search

machine can nondeterministically extend a frontier one adjacent edge at a time and return witness walks. Search policy, fairness, and resource bounds belong to that latter GSLT, not inside an opaque `reachable` relation. The already-proved functorial laws for canonical native proofs should then be strengthened from equality of their Mathlib witnesses to commuting diagrams between the retained map executions themselves.

C Work in progress: general source language and compiler

The selected edge-list materializer now establishes a narrow but complete source-language vertical: sorted constructors and two rewrites drive the generic executor, exact successor agreement connects it to a typed action function, and independent denotation connects that action to the detailed matrix machine. The remaining goal is to generalize this result to graph payload parsing, multiple conversion algorithms, and typed plan composition. That broader minimum vertical is:



Remaining theorems include canonical text/payload codec round trips, exact-image characterization, admission soundness and reflection, endpoint-safe plan composition, equality between elaborated plans and indexed GSLT paths, and a source-to-runtime operational realization. The current persistent update node must additionally be refined into address calculation and concrete cell writes before any row-major physical-cost claim. A validated vocabulary with empty rewrites satisfies none of these obligations.

D Work in progress: cost, editing, and certified solving

The detailed machines should next support local, persistent, and differential edit algorithms as competing refinements. Their structural event grades must then be related to selected physical implementations before making hardware or elapsed-time claims. Mixed revision/query workloads and maintained multi-view channels require the same treatment.

For finite-search graph properties, the intended architecture is a proved refinement into a standard solver boundary:



The solver may be untrusted; the encoding theorem and certificate replay are the trusted arrows. This boundary is not yet connected to the graph GSLTs and is not part of the proved vertical.

The remaining research programme is:

1. complete and compose the detailed matrix-to-rows and rows-to-CSR machines;

2. add terminating and confluent immediate-backtrack cancellation plus incremental trail/path admission;
3. generalize the selected operational `LanguageDef` into a canonical graph-payload codec and endpoint-safe typed plan compiler;
4. connect structural grades to selected physical implementations and measurements;
5. extend higher graphs from detailed queries to occurrence-preserving revision and construction machines;
6. connect graph-property specifications to certified SAT/DQBF/SMT/ILP encodings and replayed certificates;
7. formalize Chvátal–Erdős from separator, Menger, and longest-cycle infrastructure;
8. realize a selected workload in CeTTa and prove the complete source–semantic–runtime commuting diagram, including severance gates.

Two dangers dominate this programme. Equality of denotation must not be inflated into recovery of erased order, occurrence, provenance, witnesses, or cost. And a generated target must be the output of an actual lowering function with a commuting theorem; agreement with a separately handwritten implementation is valuable differential evidence, but it is not generation.

References

[1] The Mathlib Community. The Lean Mathematical Library. In *Proceedings of CPP 2020*, pages 367–381, 2020. doi:10.1145/3372885.3373824.

[2] Reinhard Diestel. *Graph Theory*, fifth edition. Springer, 2017. doi:10.1007/978-3-662-53622-3.

[3] Alan M. Frisch, Warwick Harvey, Christopher Jefferson, Bernadette Martínez-Hernández, and Ian Miguel. Essence: A constraint language for specifying combinatorial problems. *Constraints*, 13:268–306, 2008. doi:10.1007/s10601-008-9047-y.

[4] Çağdaş Akgün, Alan M. Frisch, Ian P. Gent, Christopher Jefferson, Ian Miguel, and Peter Nightingale. Conjure: Automatic generation of constraint models from problem specifications. *Artificial Intelligence*, 310:103751, 2022. doi:10.1016/j.artint.2022.103751.

[5] Daniele Turi and Gordon Plotkin. Towards a mathematical operational semantics. In *Proceedings of LICS 1997*, pages 280–291, 1997. doi:10.1109/LICS.1997.614955.

[6] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. doi:10.1016/0890-5401(91)90052-4.

[7] Claude Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland, 1989.

[8] G. A. Dirac. Some theorems on abstract graphs. *Proceedings of the London Mathematical Society*, s3-2(1):69–81, 1952. doi:10.1112/plms/s3-2.1.69.

[9] Oystein Ore. Note on Hamilton circuits. *The American Mathematical Monthly*, 67(1):55, 1960. doi:10.2307/2308928.

[10] L. Gregory Meredith. Computation, causality, and consciousness. Manuscript, 2026.